

AGENTIC AI & LANGGRAPH

Complete Study Notes + Interview Preparation

Agents • ReAct • Multi-Agent Systems • Deep Agents
LangGraph Architecture • Tool Engineering & Pydantic • Memory • Human-in-the-Loop

Compiled from class handwritten notes and live session transcripts

Table of Contents

1. Foundations — AI Assistant, RAG Assistant, AI Agent

- 1.1 Simple AI Assistant
- 1.2 RAG-based AI Assistant
- 1.3 What is an AI Agent
- 1.4 What is Agentic AI

2. The Agentic Flow (Think → Act → Observe)

- 2.1 Deterministic vs Dynamic Flow
- 2.2 The ReAct Loop

3. Multi-Agent Systems & Deep Agents

- 3.1 Multi-Agent (Supervisor) Architecture
- 3.2 Deep Agentic Systems
- 3.3 Shallow Agent vs Deep Agent

4. Agent Frameworks Landscape

5. Core Agentic Engineering Concepts

6. LangGraph Fundamentals

- 6.1 Why LangGraph
- 6.2 Graph, State, Nodes, Edges

7. Building a ReAct Agent in LangGraph

- 7.1 Graph Structure
- 7.2 Worked Example: 2+2 then ×10

8. Deterministic Orchestration — Supervisor / Router Pattern

9. Tool Engineering & Pydantic Schemas

10. Memory in LangGraph

- 10.1 Why Memory is Needed
- 10.2 Summary Memory & Sliding Window
- 10.3 Query Rewriting for Conversational Retrieval

11. Human-in-the-Loop (HITL)

12. Guardrails & Agent Control Parameters

13. Interview Question Bank

- Conceptual
- LangGraph & Implementation
- Scenario-based / Real-world

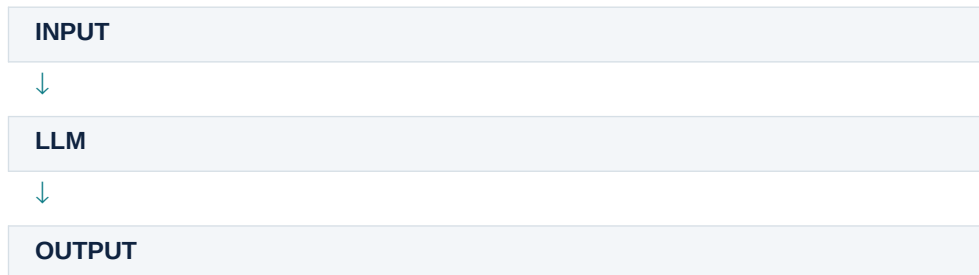
SECTION 1

Foundations

AI Assistant → RAG Assistant → AI Agent → Agentic AI

1.1 Simple AI Assistant

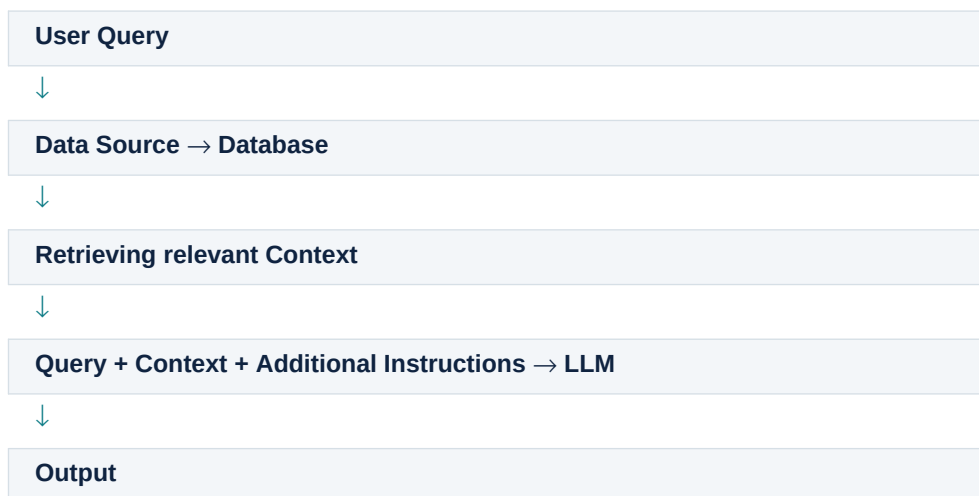
A simple AI Assistant takes an **Input**, passes it into an **LLM**, and produces an **Output**. The response is generated purely from the model's own trained (parametric) knowledge — nothing is retrieved from outside.



Example — Q: "What is Tesla?" → A: "Tesla is an EV company." (answered purely from what the model learned during training)

1.2 RAG-based AI Assistant

A **RAG Assistant** (Retrieval-Augmented Generation) is an AI assistant that retrieves relevant external knowledge from a data source and uses that information to generate a more grounded response.



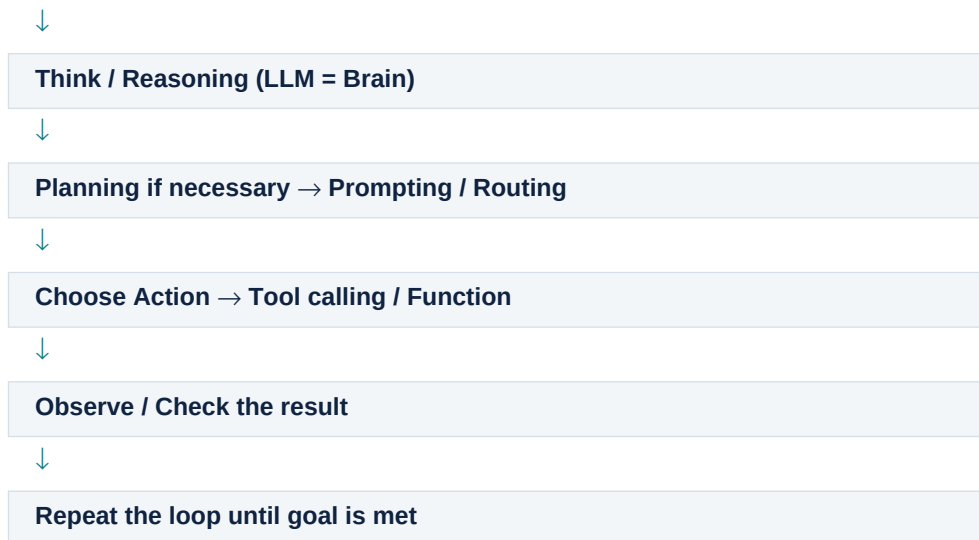
✓ Key idea

RAG upgrades a plain assistant by inserting a retrieval step before generation, so the LLM answers using up-to-date / domain-specific context instead of only its frozen training data.

1.3 What is an AI Agent

AI Agent = Autonomous System / Intelligent System. An AI Agent is an AI system that can **autonomously decide and execute actions** using tools and environmental feedback to achieve a goal. The **LLM is the "brain"** of the agent — all thinking / reasoning capability comes from the LLM.





Worked example from class — Tesla investment report

User: "Analyze Tesla and prepare an investment research report."

- I need current financial information → **Use Web Search**
- I need historical stock data → **Use Finance API**
- I need calculations → **Use Python**
- I need Tesla's annual report → **Search Documents**
- "Do I have enough information?" → No → search again; Yes → Generate Report

1.4 What is Agentic AI

Agentic AI is the umbrella term for advanced AI applications built around agents. An AI application can be a Simple Assistant, a RAG-based Assistant, or an **Agentic Assistant / Agentic System**. Agentic systems split further into:

- **Single / Mono-Agentic System** — one agent handles the whole task.
- **Multi-Agentic System** — multiple specialized agents collaborate.

■ As an AI Engineer, remember:

Agents are grouped by complexity into **Shallow Agent** vs **Deep Agent** — covered in Section 3.

SECTION 2

The Agentic Flow

Think → Act → Observe → Adapt

2.1 Deterministic Flow vs Dynamic Flow

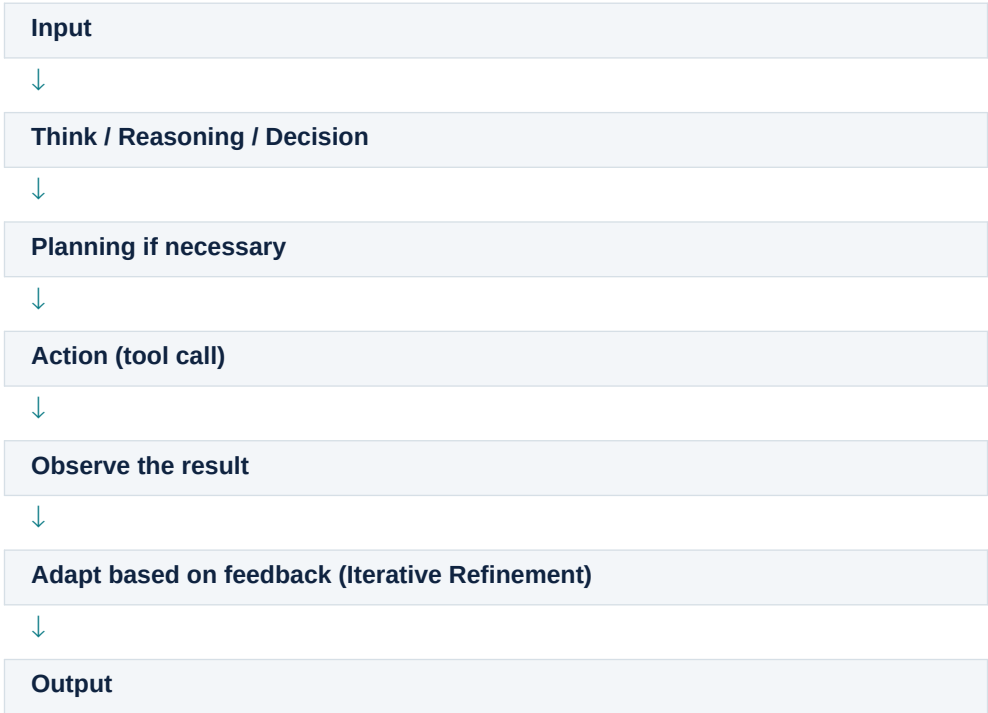
Deterministic Flow	Dynamic Flow
Routing logic between nodes is written manually by the developer	The developer writes a goal, and the LLM decides, in real time, which tool / node to call next.
Also called deterministic orchestration. A supervisor node explicitly dispatches to other nodes	The LLM dynamically takes the decision — this is the core idea behind "Explicitly dispatching the nodes"
Predictable, easy to debug, but requires the developer to anticipate every general-purpose	Flexible and general-purpose, but less predictable / harder to force a specific

■ Instructor's framing

"In the individual node specifically, we are writing logic manually in the supervisor node — deterministically we are defining everything manually. On the other hand, we can give the tool-calling capability to the LLM itself, so the LLM dynamically can make a decision." Both are still "agents" — one is deterministic orchestration, the other is dynamic (LLM-driven) orchestration.

2.2 The ReAct Loop (Reason + Act)

ReAct = **Reasoning** (LLM will think) + **Action** (tool calling). The agent repeats a loop: Input → Think → Action → Observe, looping until the goal is satisfied.



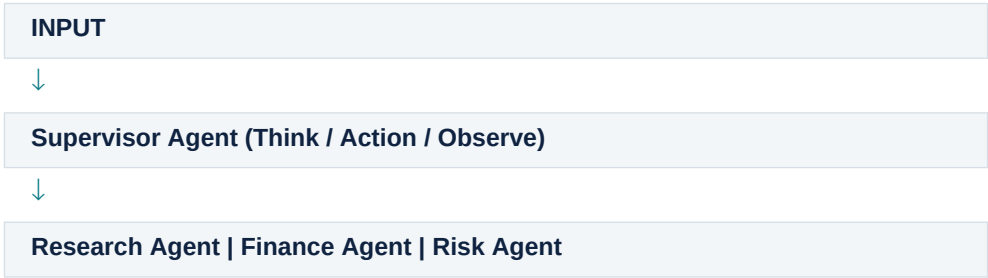
This whole loop repeats — "Repeat the process if necessary" — until the LLM decides the goal is complete and stops the loop.

SECTION 3

Multi-Agent Systems & Deep Agents

3.1 Multi-Agent System

A **Multi-Agent System** uses multiple agents instead of one agent. **Each agent specializes in a particular job.** A **Supervisor Agent** receives the input, reasons about it (Think/Action/Observe), and delegates sub-tasks to specialized worker agents.



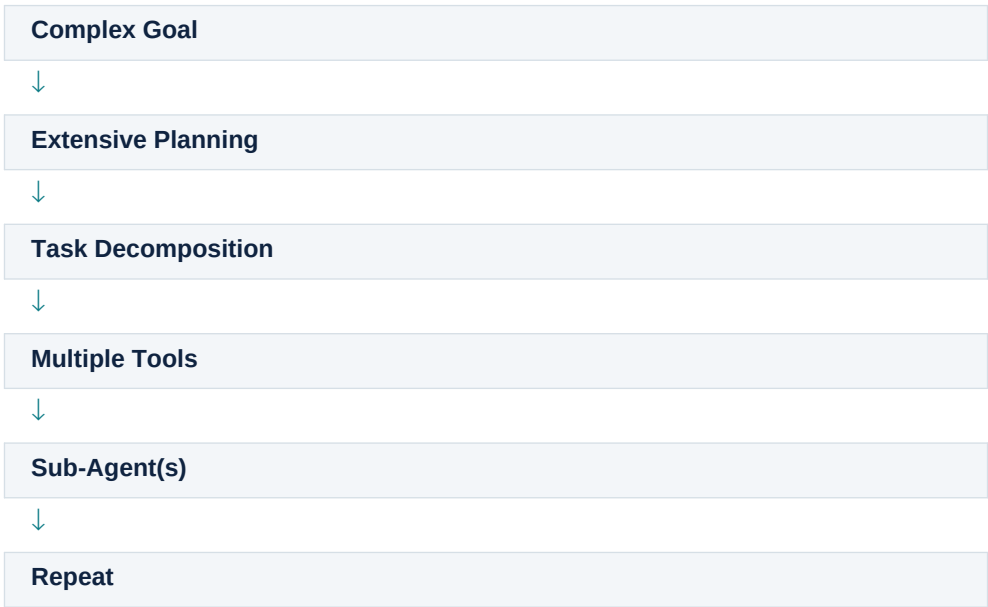
Tesla example — supervisor delegation

- **Research Agent** → Tesla products, Market, Competitors, Recent developments
- **Financial Agent** → Revenue, Profit, Cash flow, Margins
- **Risk Agent** → Regulatory risks, Competition, Supply-chain risks
- **Market Agent** → EV market, Competitors, Market share
- **Writer Agent** → combines everything into the final report

Instead of...	...we now have
One Agent does everything (research, finance, risk, writing)	Multiple Specialized Agents that collaborate / delegate and then

3.2 Deep Agentic System

A **Deep Agent** is built for **large, long-running, complex problems** that need **extensive planning, task decomposition** and possibly multiple tools / sub-agents, executed iteratively toward a final deliverable.





Final Deliverable

Tesla example — Deep Agent execution

A Deep Agent could first generate a **TODO plan**:

- 1. Download Tesla reports 2. Extract financial data 3. Research competitors
- 4. Analyze EV market 5. Calculate financial ratios 6. Generate charts
- 7. Research risks 8. Compare companies 9. Verify sources 10. Write report

...then work through those tasks, creating a working file structure, e.g.:

```
/workspace
  /tesla_financials.csv
  /competitor_analysis.md
  /market_research.md
  /risk_analysis.md
  /charts/
  /sources.md
  /final_report.docx
```

If context becomes too large: **Large Research** → **Summarize / Store in Files** → **Keep only relevant context** → **Continue Working**. A Deep Agent may also delegate to sub-agents, e.g. Financial Research Sub-agent, Competitor Research Sub-agent, Market Research Sub-agent, Verification Sub-agent.

3.3 Shallow Agent vs Deep Agent

As an AI Engineer you classify agent designs on a spectrum: **Shallow Agent** (single/mono-agentic system, simple task) at one end, **Deep Agent** (large & long-running, complex problem — extensive planning, multiple steps, long task execution, possibly multi-agentic underneath) at the other.

SECTION 4

Agent Frameworks Landscape

Ways an agentic system can be implemented, as listed in class notes:

#	Approach	Backed By
1	Pure Python	From scratch
2 ★ winner	LangGraph	LangChain — 1st most popular
3	OpenAI Agent SDK	OpenAI — 2nd most popular
4	Google ADK	Google — 2nd most popular
5	Autogen	Microsoft — 3rd most popular
6	CrewAI	3rd most popular
7	SmolAgent	HuggingFace
8	n8n	No-code framework

■ Why the course uses LangGraph

Marked in the handwritten notes as the ★ "winner" / primary framework taught in this course — the instructor builds all hands-on examples (ReAct agent, supervisor/router, memory, human-in-the-loop) in LangGraph, and only gives a "glimpse" of Google ADK / AutoGen afterwards.

SECTION 5

Core Agentic Engineering Concepts

Capabilities an agent design must account for (from class syllabus):

1. Planning / Dynamic planning
2. Execution
3. Task decomposition
4. Reflection
5. Self-correction / improvement / healing / critic
6. Reasoning
7. Parallelization
8. Orchestration
9. Human in the loop
10. Feedback-aware optimization

"It is related to the Agentic Flow." — all ten map back onto the Think/Act/Observe loop and multi-agent orchestration discussed above.

Engineering skills required to build production agents

- | | |
|---------------------------------|------------------------------------|
| 1. Pydantic enum type hints | 8. Prompting |
| 2. JSON schema | 9. Context engineering |
| 3. Async Python | 10. MCP A2A Protocol |
| 4. Structured Output | 11. Evaluation / testing of agents |
| 5. Tool engineering | 12. Observability |
| 6. Memory management | 13. Guardrails Security |
| 7. State management | 14. Caching |

SECTION 6

LangGraph Fundamentals

6.1 LangGraph vs LangChain vs LangSmith

The course syllabus for LangGraph begins by distinguishing LangGraph from LangChain and LangSmith, and comparing a LangGraph program against writing the same logic in normal Python code — LangGraph gives you graph-based structure (nodes, edges, state) instead of hand-rolled control flow.

6.2 LangGraph Topic Roadmap (as taught)

- | | |
|--|----------------------|
| 1. Introduction of LangGraph | 8. Memory |
| 2. LangGraph Architecture & tools | 9. Human in the loop |
| 3. State management | 10. Streaming |
| 4. StateGraph API vs Graph API vs Functional API | 11. Multi-agent |
| 5. Send API() map-reduce | 12. Caching |
| 6. Command API() | 13. Fault tolerance |
| 7. Persistence / Checkpoint | 14. Async LangGraph |

■ Graph visualization note

Send API / map-reduce and Command API are grouped together with **Subgraphs** in the handwritten notes — these are the mechanisms LangGraph offers for fan-out/fan-in and dynamic control transfer between nodes.

Core building blocks

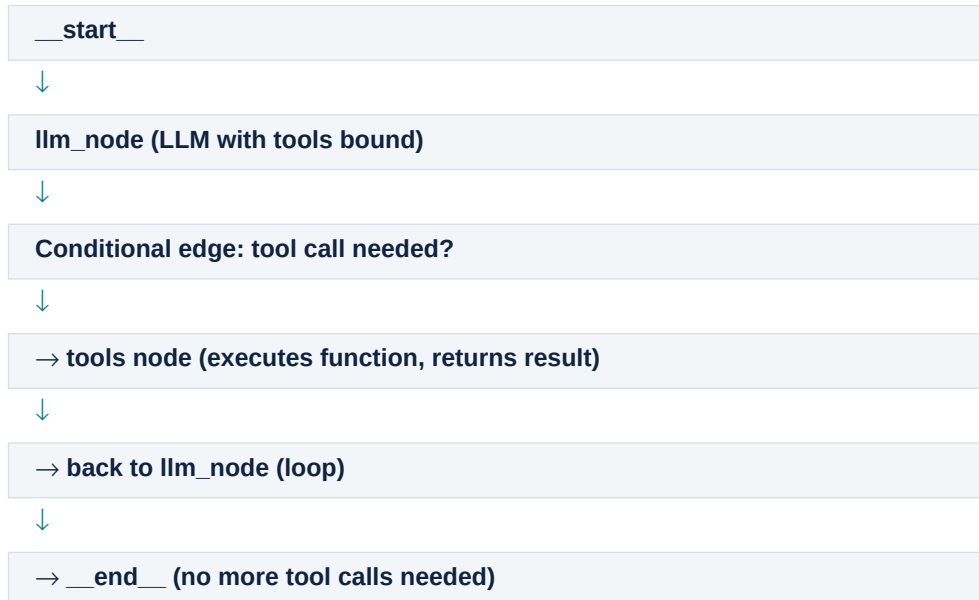
- **State** — the shared data object passed between nodes (e.g. question, route, context, draft_answer).
- **Node** — a function (or an LLM call) that reads state and returns updates to it.
- **Edge** — connects nodes; can be a fixed edge or a **conditional edge** that routes dynamically based on state.
- **Checkpoint** (e.g. InMemorySaver) — persists a snapshot of graph state, enabling resumability and human-in-the-loop interrupts.

SECTION 7

Building a ReAct Agent in LangGraph

7.1 Graph structure

The classic ReAct graph in LangGraph: `__start__` → `llm_node` (LLM bound with tools — `llm_with_tool`) → conditionally routes to `tools` node or `__end__`. The key question the graph is answering at each step: *"How is the LLM going to decide which tool needs to be called?"*



Example tool set bound to the LLM: [Add, Sub, Mul, Div, Pow]

7.2 Worked example — "What is 2+2 and multiply the output of it with 10?"

- Prompt + Reasoning → sent to `llm_with_tool`
- LLM decides: first call **Add-tool** with a=2, b=2 → returns **4**
- LLM takes that output and decides: call **Mul-tool** with a=4, b=10 → returns **40**
- No further tool needed → **--end--** → final answer: **40**

✓ What this demonstrates

This is **dynamic flow** in action — the LLM is dynamically deciding, at each step, which tool to call and with what arguments, based on the running conversation/tool output, rather than the developer hard-coding "call Add, then call Mul".

Tool Engineering view

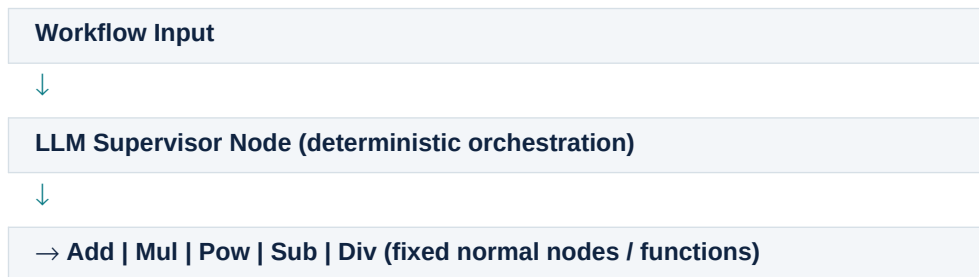
From the LLM's perspective: the LLM is given a set of **Tools** (with their required parameters) and must decide which tool to call. The chosen tool executes as a **Tool Node (function)**, e.g. Add / Mul / Div / Sub — each is described in the notes as a **"normal node (function)"** that a **Supervisor LLM** or the reasoning LLM itself can dispatch to.

SECTION 8

Deterministic Orchestration — Supervisor / Router Pattern

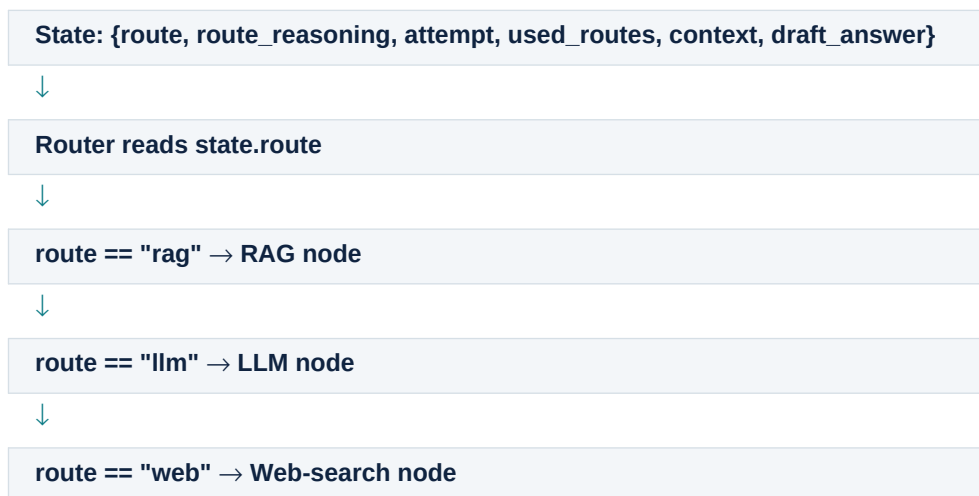
An alternative to letting the LLM freely decide (ReAct/dynamic) is a **deterministic workflow**: a Supervisor / Router node inspects the state and manually routes to one of several fixed nodes. Both approaches are "agents" — the difference is who decides the next step (developer-written logic vs the LLM).

Example: Supervisor LLM routing to arithmetic nodes



Example: RAG / LLM / Web router built live in class

The router node reads several fields out of the shared **state**: `route`, `route_reasoning`, `attempt`, `used_routes`, `context`, `draft_answer`. It takes the value of `route` (which can be `rag`, `llm`, or `web`), matches it like a dictionary key, and dispatches execution to the correspondingly named node.



Deterministic vs Dynamic — instructor's summary

- **Deterministic Agent:** "This is also my agent, but it's a deterministic agent. It's a deterministic flow... I would like to say this is my orchestrating, deterministic orchestration, where the LLM is one of the — like, where the LLM is also part of the process."
- **Dynamic Agent (ReAct-style):** the LLM itself is given tool-calling capability and dynamically makes the decision of which node/tool runs next — no manual "if/else" routing needed inside the workflow.
- "Any sort of a functionality you can write inside the tool" — tools are just plain functions; what varies is who decides to call them.

SECTION 9

Tool Engineering & Pydantic Schemas

Tool engineering is about precisely describing a function's inputs so the LLM (and the runtime) know exactly what to pass. This matters because an LLM calling a tool with the wrong data type (e.g. a string instead of a float) can break execution.

*args vs **kwargs (why tool signatures get tricky for an LLM)

- `*args` → a **tuple** of positional values; can accept any number of arguments.
- `**kwargs` → a **dictionary** of keyword arguments, e.g. `{name: "Sunny", age: 30, city: "Bangalore"}`.
- For a tool like a **calculator**, you must be explicit: e.g. the numeric argument should be typed as `float`, not left ambiguous — otherwise the LLM cannot reliably know what data type to pass.

■ Instructor's guidance

"You should have some, basically, what I can say — you will have to understand what we are going to pass to that particular parameter/function... the data type of that particular argument you will be mentioning... define the Pydantic schema — in that case it will be working."

Pydantic model basics (from live demo)

Pydantic models let you define required and optional fields, with default values and type validation, for tool inputs / structured output. Demonstrated behaviour:

```
class User(BaseModel):
    name: str
    country: str = "India"          # default value
    middle_name: str | None = None  # optional, defaults to None
    nickname: str | None = None     # optional, defaults to None

User(name="Sunny", country="USA", middle_name="Kumar", nickname="Sun")
# -> works: name=Sunny, country=USA, middle_name=KUMAR, nickname=Sun

User(name="Sunny", middle_name="KUMAR")
# -> also works: country defaults to "India", nickname defaults to None
```

- If a field is not passed and it has a default, Pydantic fills in the default (e.g. `country` defaults to `"India"`).
- `None` means "nothing" — used as the default for genuinely optional fields like `middle_name` / `nickname`.
- This same schema discipline is what you apply to **tool arguments** so the LLM's tool call is validated and type-safe.

SECTION 10

Memory in LangGraph

10.1 Why memory is needed

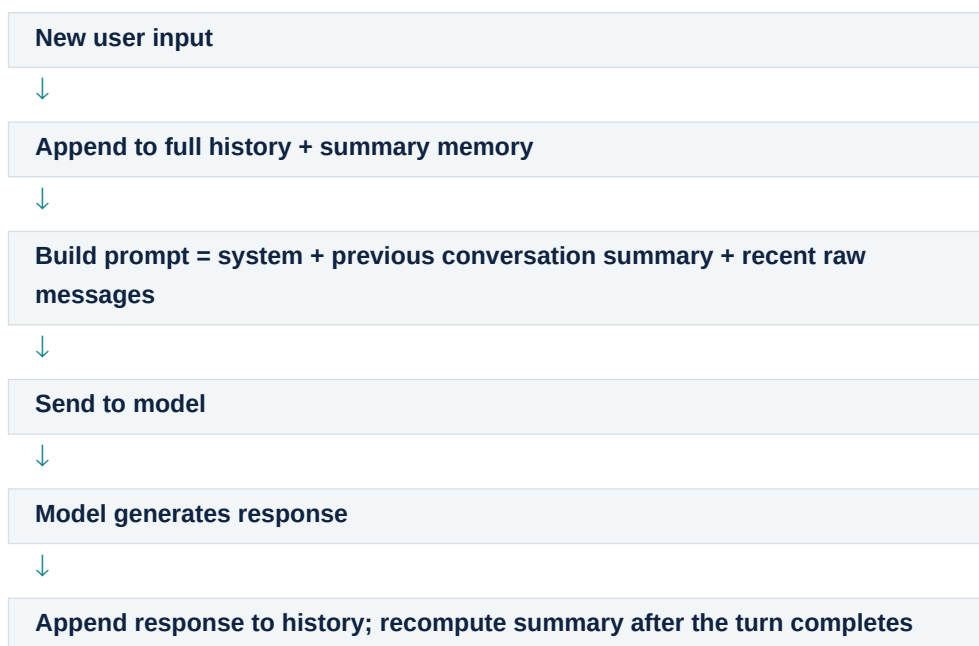
Without memory, every turn of the conversation is stateless — the agent forgets prior turns. LangGraph memory keeps track of conversation history (and a compressed summary of older turns) so multi-turn conversations stay coherent without unboundedly growing the prompt sent to the LLM.

10.2 Summary Memory + Sliding Window

A conversational memory class built live in class exposes methods:

- `create_summary` — build an initial summary of older turns.
- `compress_summary` — fold new older messages into the running summary.
- `add_user_message` / `add_assistant_message` — append new turns into memory.
- `get_recent_conversation` — returns the most recent raw messages, or "no recent conversation" if memory is empty.

The compression rule: **"compress only after a complete user–assistant turn"** — i.e. summarization is triggered once a full exchange (user message + assistant reply) has completed, not mid-turn.



■ Sliding-window detail from Q&A;

In the demoed design, the **older summary** captures context from earlier messages (e.g. summarized from the first 4 messages), while the most recent messages (e.g. the last 6) are kept as **raw recent history** rather than summarized. As the conversation grows, the window slides forward: older raw messages get folded into the summary, and the newest ones stay verbatim — this keeps prompt size bounded while preserving recent detail.

✓ Why not just keep everything raw?

"If we are taking directly those recent messages, then we don't require the summary, right? We don't need [it]." — summarization is specifically needed once history grows too large to keep passing in full.

10.3 Query Rewriting (for conversational retrieval)

A `rewrite_query` method turns a follow-up question into a **standalone query** using recent conversation history, so retrieval doesn't depend on unresolved pronouns / context from earlier turns:



This is a standard conversational-RAG technique: resolve context-dependent follow-ups ("what about the second one?") into a fully self-contained query before hitting the retriever.

SECTION 11

Human-in-the-Loop (HITL)

Human-in-the-loop lets a LangGraph workflow **pause execution** at a sensitive step and wait for a human decision before continuing — critical for high-stakes actions (e.g. financial refunds) where full autonomy is undesirable.

Worked example — customer refund approval



Demo input: "Customer101 received a damaged product, please refund that." The graph runs until `human_approval`, then **interrupts** and prints details for review: action = refund, customer ID, amount, region — along with the message "Graph paused for human approval."

Implementation building blocks

- **Checkpoint** (`InMemorySaver`) — saves a snapshot of graph state in memory so the graph can be resumed after the interrupt.
- **Compile the graph** with the checkpointer attached.
- **Config** — includes a `thread_id` that identifies this particular run/conversation, so the correct paused state is resumed.
- **approve_router** — a conditional-edge function that reads the human's decision and routes to `execute` or `end` (`reject`).
- Resuming: capture the human's yes/no input, pass it back into the graph (e.g. via a `Command`) using the same thread config, and the graph continues from exactly where it paused.

■ Why this matters

HITL turns an otherwise fully autonomous agent into a **supervised** one for specific risky actions — the graph literally halts mid-execution and cannot proceed until a human signal is received, which is the safest way to gate consequential agent actions (payments, deletions, external emails, etc.).

SECTION 12

Guardrails & Agent Control Parameters

To keep an agent loop from running away (infinite loops, runaway cost), production agent systems enforce hard limits:

- **Maximum iteration limit** — caps how many Think→Act→Observe cycles the agent may run.
- **Timeout** — caps wall-clock time for a run.
- **Stop condition** — explicit condition(s) that end the loop early once satisfied.
- **Token / cost budget** — caps spend (tokens or \$) per run.

These sit alongside the broader engineering checklist from Section 5 — Guardrails | Security and Observability are treated as first-class production concerns, not afterthoughts, once an agent can call tools and loop autonomously.

SECTION 13

Interview Question Bank

Prepared strictly from the concepts covered in these notes

A. Conceptual — Agentic AI Fundamentals

1

What is the difference between a simple AI Assistant and a RAG-based AI Assistant?

A simple AI Assistant answers purely from the model's trained knowledge (Input → LLM → Output). A RAG Assistant first retrieves relevant external context from a data source / database for the user's query, then passes Query + Context + instructions into the LLM, producing a more grounded, up-to-date answer.

2

How is an AI Agent different from a simple AI Assistant?

An assistant only generates text responses. An AI Agent is an autonomous, intelligent system that can decide and execute actions using tools and environmental feedback to achieve a goal — it thinks/reasons (LLM = brain), plans if necessary, chooses and calls tools, observes the result, and loops until the goal is met.

3

What is the role of the LLM inside an agent?

The LLM is the "brain" of the agent — all reasoning, planning, and decision-making (which tool to call, when to stop) originates from the LLM. Tools themselves are just functions; the LLM decides when and how to use them.

4

Explain the ReAct pattern.

ReAct = Reasoning + Action. The LLM reasons about the next step, takes an action (calls a tool), observes the result, and repeats this Think→Act→Observe loop until it has enough information to produce a final answer. It's a dynamic flow because the LLM decides each next step at runtime rather than following a hard-coded path.

5

What is the difference between deterministic flow and dynamic flow in agent design?

Deterministic flow means the developer manually writes routing logic between nodes (a supervisor decides the next node explicitly, if/else style). Dynamic flow means the LLM itself is given tool-calling capability and dynamically decides at runtime which tool/node to invoke next — no hard-coded routing required.

6

What is a Multi-Agent System and why use one?

A Multi-Agent System uses multiple specialized agents instead of one agent doing everything. A Supervisor Agent receives the input and delegates sub-tasks to specialized worker agents (e.g. Research, Financial, Risk, Market, Writer agents for an investment report), then combines their outputs into the final deliverable. This improves modularity, specialization, and manageability versus one monolithic agent.

7

What is a Deep Agent, and when would you use one?

A Deep Agent is designed for large, long-running, complex problems that require extensive planning, task decomposition, multiple tools, and possibly sub-agents, executed iteratively toward a final deliverable (e.g. generating a TODO plan, working through it step by step, writing intermediate files to a workspace, and summarizing/storing large research results to keep context manageable).

8

What's the difference between a Shallow Agent and a Deep Agent?

A Shallow Agent handles a simple task with a single/mono-agentic system. A Deep Agent tackles a complex problem needing extensive multi-step planning and long task execution, often orchestrating multiple sub-agents or tools before producing a final deliverable.

9

List the core capabilities an agentic system needs to support.

Planning/dynamic planning, execution, task decomposition, reflection, self-correction/improvement/healing/critic, reasoning, parallelization, orchestration, human-in-the-loop, and feedback-aware optimization.

10

Name popular frameworks for building agents and how they compare.

Pure Python (from scratch), LangGraph (LangChain — most widely used / taught in this course), OpenAI Agent SDK (OpenAI), Google ADK (Google), Autogen (Microsoft), CrewAI, SmolAgent (HuggingFace), and n8n (no-code framework).

B. LangGraph & Implementation

11

What are the core building blocks of a LangGraph graph?

State (shared data passed between nodes), Nodes (functions or LLM calls that read/update state), Edges (fixed or conditional connections between nodes), and a Checkpointer (persists state snapshots, enabling resumability and human-in-the-loop).

12

Describe the graph structure of a typical ReAct agent in LangGraph.

`__start__` → `llm_node` (LLM bound with tools) → a conditional edge checks whether a tool call is needed: if yes, go to the tools node, execute it, and loop back to `llm_node`; if no more tool calls are needed, go to `__end__`.

13

Walk through the worked example: "What is 2+2 and multiply the output with 10?"

The LLM first calls the Add tool with `a=2`, `b=2`, getting 4. It then calls the Mul tool with `a=4`, `b=10`, getting 40. Since no further tool is needed, the graph ends and returns 40 — demonstrating the LLM dynamically chaining tool calls based on prior outputs.

14

How does a router/supervisor node decide which node to call next?

It reads fields out of shared state — e.g. `route`, `route_reasoning`, `attempt`, `used_routes`, `context`, `draft_answer` — and switches on the value of 'route' (e.g. 'rag', 'llm', 'web'), dispatching to the correspondingly named node, similar to a dictionary key lookup.

15

Why can tool arguments be tricky for an LLM to generate correctly?

Because tool functions can accept `*args` (a tuple of positional values) or `**kwargs` (a dictionary of named values) with ambiguous types. Without an explicit schema (e.g. Pydantic model specifying a numeric field as float), the LLM may pass the wrong data type, breaking the tool call.

16

Why use Pydantic when defining tool schemas?

Pydantic lets you declare required vs optional fields, default values, and strict types for each argument, so tool calls are validated automatically — reducing runtime errors from malformed or mistyped LLM-generated arguments.

17

What is the purpose of a checkpointer like InMemorySaver in LangGraph?

It persists a snapshot of the graph's state so execution can be paused and resumed later — essential for human-in-the-loop flows where the graph must interrupt and wait for a human decision, then continue exactly where it left off using the same `thread_id` config.

18

How does LangGraph's human-in-the-loop pattern work end to end?

The graph runs normally (e.g. start → analyze) until it hits a `human_approval` node, where it interrupts and surfaces the proposed action for review. A human supplies a decision (e.g. yes/no); this is passed back into the graph (e.g. via a `Command`) using the same `thread_id`, and an `approve_router` conditional edge routes to execute or reject/end accordingly.

19

How would you implement multi-turn memory in a LangGraph-based chatbot without letting the prompt grow unbounded?

Use a summary-memory design: keep the most recent `N` raw messages verbatim, and periodically compress older messages into a running summary (only after a complete user-assistant turn). Each new call builds the prompt as system + summary + recent raw messages, keeping the total size bounded while retaining conversational context.

20

What is query rewriting and why is it used before retrieval in a conversational RAG agent?

Query rewriting converts a context-dependent follow-up question (e.g. "what about the second one?") into a fully self-contained standalone query, using recent conversation history. This standalone query is then used for retrieval, since the retriever has no access to prior conversational context on its own.

21

What guardrail parameters should a production agent enforce, and why?

Maximum iteration limit and timeout (prevent infinite/runaway loops), stop conditions (end the loop as soon as the goal is met), and token/cost budgets (cap spend per run) — all needed because an autonomous Think→Act→Observe loop could otherwise run indefinitely or become very expensive.

C. Scenario-based / Real-world

22

A student is building an internal chatbot on Azure DevOps using a ReAct agent (create_react_agent) with ~30 tools bound to it (search, create/update work items, task decomposition, chart generation). As more tools are added, what problems would you expect, and how would you address them?

Binding many tools directly to one ReAct agent increases the chance of the LLM choosing the wrong tool, bloats the prompt (all tool schemas are in context), and increases latency/cost per call. This is a strong signal to move toward a multi-agent / supervisor pattern: split tools across specialized sub-agents (e.g. a 'work item' agent, a 'reporting' agent), and use a supervisor/router node to deterministically or dynamically delegate to the right specialist — reducing the tool surface any single LLM call has to reason over.

23

You need an agent to autonomously refund customers, but the business wants a human to approve any refund above a threshold. How would you design this in LangGraph?

Use the human-in-the-loop pattern: build a graph with an analyze node that decides if approval is required, a human_approval node where the graph interrupts (using a checkpoint + thread_id so it can be resumed), and an approve_router conditional edge that routes to execute or reject based on the human's yes/no response passed back into the graph.

24

A RAG system over 200–300 page annual reports keeps missing table/numeric data points. What's the likely issue and how would you fix it?

Chunking the entire report as generic text loses the structure of tables and numeric data points, so semantically-similar-but-wrong chunks get retrieved. Fix: extract tables separately and embed them distinctly (or use hybrid retrieval + metadata filtering to narrow the search space to the right section before running semantic search), rather than relying purely on top-k similarity search over the whole document.

25

When would you choose a deterministic (supervisor-routed) workflow over a fully dynamic ReAct agent?

When you need predictability, auditability, or tight control over cost/latency for a well-understood task with a small number of known branches (e.g. routing to rag/llm/web based on a classified intent). ReAct-style dynamic tool-calling is preferable when the task space is open-ended and you can't enumerate every path in advance, at the cost of less predictable execution.

26

How would you decide between a single (mono) agent and a multi-agent architecture for a new use case?

Start with a single agent for simple, narrow tasks. Move to multi-agent / supervisor architecture once the task naturally decomposes into distinct specialties (e.g. research vs financial analysis vs risk vs writing), the tool count grows too large for one agent to reason over reliably, or you need long-running / deep-agent style planning with sub-agents handling independent chunks of work.

All content in this document was compiled exclusively from the user-provided handwritten class notes (PDF scans) and Zoom class-recording transcripts (VTT). No external sources were used.